

CAOS MALAYALM

Block 1: Basic Functional Architecture

ബ്ലോക്ക് 1: അടിസ്ഥാന പ്രവർത്തന ഘടന

1.1.1 കമ്പ്യൂട്ടറുകളുടെ തലമുറകൾ ★★

പരിക്ഷയ്ക്ക്:

അഞ്ച് തലമുറകളും അവയുടെ കാലഘട്ടം, സാങ്കേതികവിദ്യ, ഒരു പ്രത്യേകത, ഒരു ഉദാഹരണം എന്നിവ എഴുതുക.

1. ഒന്നാം തലമുറ (1940–1956)

- സാങ്കേതികവിദ്യ: **Vacuum Tubes, Magnetic Drums**
- വളരെ വലുതും ചെലവേറിയതുമായിരുന്നു.
- വളരെ അധികം ചൂട് ഉൽപ്പാദിപ്പിച്ചിരുന്നു.
- ഉദാഹരണങ്ങൾ: **UNIVAC, ENIAC, EDVAC**

2. രണ്ടാം തലമുറ (1956–1963)

- സാങ്കേതികവിദ്യ: **Transistors**
- വലുപ്പം കുറവ്, വേഗത കൂടുതൽ, ചൂട് കുറവ്.
- **COBOL, FORTRAN** പോലുള്ള High-level languages ഉപയോഗിച്ചു.
- ഉദാഹരണങ്ങൾ: **IBM 1400 series, IBM 7090**

3. മൂന്നാം തലമുറ (1964–1971)

- സാങ്കേതികവിദ്യ: **Integrated Circuits (ICs)**
- കൂടുതൽ വിശ്വസനീയവും വേഗതയേറിയതും ചെറുതും വിലകുറഞ്ഞതുമായിരുന്നു.
- ഉദാഹരണങ്ങൾ: **IBM System/360, PDP-11**

4. നാലാം തലമുറ (1971–2010)

- സാങ്കേതികവിദ്യ: **VLSI / Microprocessors**
- Personal Computers, Laptops എന്നിവ വികസിച്ചു.
- ഉദാഹരണങ്ങൾ: **Apple, CRAY-1**

5. അഞ്ചാം തലമുറ (നിലവിലും ഭാവിയ്ക്കും)

- സാങ്കേതികവിദ്യ: **Artificial Intelligence (AI)**
- മനുഷ്യരെപ്പോലെ ചിന്തിക്കുകയും പ്രവർത്തിക്കുകയും ചെയ്യാൻ ശ്രമിക്കുന്ന സംവിധാനങ്ങൾ.
- ഉദാഹരണം: **PARAM 10000**

കൂടാതെ എഴുതുക:

von Neumann-ന്റെ അഞ്ച് യൂണിറ്റുകൾ — **Input, Output, Memory, ALU, Control** — എല്ലാ തലമുറകളിലും അടിസ്ഥാനപരമായി നിലനിൽക്കുന്നു.

1.1.2 കമ്പ്യൂട്ടറിന്റെ Functional Units ★★

പരിക്ഷയ്ക്ക്:

von Neumann-ന്റെ പരിചയം, അഞ്ച് യൂണിറ്റുകളും അവയുടെ പ്രവർത്തനവും, അവ തമ്മിൽ എങ്ങനെ പ്രവർത്തിക്കുന്നു, CPU-യുടെ നിർവ്വചനം, Block Diagram എന്നിവ ഉൾപ്പെടുത്തുക.

Introduction:

John von Neumann-ന്റെ **Stored-Program Model** അനുസരിച്ച് ഒരു കമ്പ്യൂട്ടറിൽ അഞ്ച് പ്രധാന Functional Units ഉണ്ട്:

1. Input Unit
2. Output Unit
3. Memory Unit
4. ALU
5. Control Unit

CPU = Control Unit + ALU + Registers

1. Input Unit

- Data-യും programs-ഉം സ്വീകരിക്കുന്നു.
- Keyboard, Mouse, Scanner, Disk തുടങ്ങിയ ഉപകരണങ്ങളിൽ നിന്ന് data സ്വീകരിക്കുന്നു.
- Data-യെ **Binary form**ലേക്ക് മാറ്റുന്നു.

2. Memory Unit

- Instructions, Data, Results എന്നിവ സംഭരിക്കുന്നു.
- **Primary Memory (RAM/ROM)** വേഗതയേറിയതാണ്.

- **Secondary Memory (Disk)** സ്ഥിരമായ storage ആണ്, എന്നാൽ വേഗത കുറവാണ്.
- ഒരു program execute ചെയ്യുമ്പോൾ അത് **Main Memory**യിൽ ഉണ്ടായിരിക്കണം.

3. ALU — Arithmetic Logic Unit

- Arithmetic operations ചെയ്യുന്നു:
 - 
 - 
 - 
 - 
- Logical, comparison operations ചെയ്യുന്നു.
- CPU-യുടെ പ്രധാന **work area** ആണ്.

4. Control Unit

- കമ്പ്യൂട്ടറിന്റെ **nerve centre** ആണ്.
- Instructions **fetch** ചെയ്യുകയും **interpret** ചെയ്യുകയും ചെയ്യുന്നു.
- മറ്റ് യൂണിറ്റുകൾക്ക് **control signals** നൽകുന്നു.
- Timing നിയന്ത്രിക്കുന്നു.

5. Output Unit

- Processing-ന്റെ ഫലം ഉപയോക്താവിന് കാണിക്കുന്നു.
- ഉദാഹരണങ്ങൾ:
 - Monitor
 - Printer
 - Speaker

എല്ലാ യൂണിറ്റുകളും ഒരുമിച്ച് പ്രവർത്തിക്കുന്നത്

Input → Memory → Control Unit → Fetch–Decode–Execute → ALU → Result → Memory / Output

ഓരോ യൂണിറ്റിനും വ്യത്യസ്തമായ പ്രവർത്തനമുണ്ട്. എന്നാൽ ഇവയുടെ **coordinated operation** വഴിയാണ് ഒരു കമ്പ്യൂട്ടർ പൂർണ്ണമായി പ്രവർത്തിക്കുന്നത്.

1.1.2.2 Primary and Secondary Memory ★★

Primary Memory

1. വേഗതയേറിയ semiconductor storage ആണ്.
2. **RAM, ROM** എന്നിവ ഇതിൽപ്പെടുന്നു.
3. RAM **volatile** ആണ്.
4. ROM **non-volatile** ആണ്.
5. Disk-നെ അപേക്ഷിച്ച് ചെലവേറിയതാണ്.
6. Program പ്രവർത്തിക്കുമ്പോൾ അത് Primary/Main Memory-യിൽ ഉണ്ടായിരിക്കണം.
7. ഉദാഹരണങ്ങൾ: **RAM, ROM**

Secondary Memory

1. Primary memory-യെക്കാൾ വേഗത കുറവാണ്.
 2. വില കുറവാണ്.
 3. Permanent storage നൽകുന്നു.
 4. **Non-volatile** ആണ്.
 5. വലിയ അളവിലുള്ള data സംഭരിക്കാൻ ഉപയോഗിക്കുന്നു.
 6. ഉദാഹരണങ്ങൾ: **Hard Disk, Optical Disk**
-

1.1.4 Bus Types ★★

Bus: CPU, Memory, Peripherals എന്നിവ തമ്മിൽ ബന്ധിപ്പിക്കുന്ന shared set of lines ആണ്.

1. Internal / System Bus

Processor, Memory, I/O എന്നിവ തമ്മിൽ ബന്ധിപ്പിക്കുന്നു.

2. External Bus

Keyboard, Mouse തുടങ്ങിയ peripherals-നെ CPU-യുമായി ബന്ധിപ്പിക്കുന്നു.

System Bus-ന്റെ മൂന്ന് പ്രധാന ഭാഗങ്ങൾ

1. Data Bus

- Bidirectional ആണ്.
- Data കൈമാറുന്നു.

2. Address Bus

- Unidirectional ആണ്.
- Address കൈമാറുന്നു.

3. Control Bus

- Read/Write പോലുള്ള control signals കൈമാറുന്നു.

Lines **dedicated** ആയിരിക്കാം അല്ലെങ്കിൽ **multiplexed** ആയിരിക്കാം. സാധാരണ I/O connection ഒരു **single/common bus** ഉപയോഗിക്കുന്നു.

ബ്ലോക്ക് 2: I/O and DMA

2.1 RTL and Common Bus ★★

RTL — Register Transfer Language

RTL (Register Transfer Language) എന്നത് registers തമ്മിലുള്ള data transfer, micro-operations എന്നിവ രേഖപ്പെടുത്താൻ ഉപയോഗിക്കുന്ന ഭാഷയാണ്.

ഉദാഹരണം:

$R2 \leftarrow R1$

ഇതിന്റെ അർത്ഥം: Control signal enable ചെയ്താൽ **R1-ലെ data R2-ലേക്ക് copy ചെയ്യുന്നു.**

Fetch Cycle

ഒരു instruction fetch ചെയ്യുമ്പോൾ സാധാരണ നടക്കുന്ന ഘട്ടങ്ങൾ:

1. **$MAR \leftarrow PC$**
2. Memory Read ചെയ്യുന്നു; Instruction $\rightarrow$ **IR**
3. **$PC \leftarrow PC + 1$**

Common Bus-ലെ Multiplexer

Common bus-ൽ പല registers-നും ഒരേ bus share ചെയ്യാൻ **Multiplexer** ഉപയോഗിക്കുന്നു.

- ഏത് register-ന്റെ data bus-ലേക്ക് വരണം എന്ന് Multiplexer തിരഞ്ഞെടുക്കുന്നു.

- ഇത് **Select Control Signals** ഉപയോഗിച്ചാണ് ചെയ്യുന്നത്.

2.2 Functions of I/O Interface ★★

CPU-യും peripheral devices-ഉം തമ്മിൽ speed, data format, operation എന്നിവയിൽ വ്യത്യാസമുണ്ട്.

I/O Interface എന്നത് Processor Bus-നും I/O Device-നും ഇടയിൽ പ്രവർത്തിക്കുന്ന ഒരു interface ആണ്.

1. Data Transfer

CPU/Memory-യും device-ഉം തമ്മിൽ data കൈമാറുന്നു.

- Ports / Registers ഉപയോഗിച്ചാണ് data transfer ചെയ്യുന്നത്.

ഉദാഹരണം: Keyboard-ൽ നിന്ന് CPU-വിളേക്ക് data നൽകുന്നത്.

2. Control Signal Management

Peripheral device-നെ:

- Start ചെയ്യുക
- Stop ചെയ്യുക
- Coordinate ചെയ്യുക

എന്നിവയ്ക്കുള്ള control signals കൈകാര്യം ചെയ്യുന്നു.

Control / Status / Data-in / Data-out signals ഉപയോഗിക്കുന്നു.

3. Data Conversion

CPU-യിൽ നിന്നുള്ള data format അല്ലെങ്കിൽ signal format device-ന് അനുയോജ്യമായ രീതിയിലേക്ക് മാറ്റുന്നു.

ഉദാഹരണം:

Parallel CPU data ↔ Serial UART data.

4. Synchronisation

വേഗതയേറിയ CPU-വും slow peripheral device-ഉം തമ്മിൽ coordination ഉണ്ടാക്കുന്നു.

ഇതിനായി:

- Status

- Handshake
- Buffers

എന്നിവ ഉപയോഗിക്കുന്നു.

ഇത് data നഷ്ടപ്പെടുന്നത് തടയുന്നു.

Polling

CPU device status വീണ്ടും വീണ്ടും പരിശോധിക്കുന്നു.

പ്രയോജനം: Simple

ദോഷം: CPU time waste ചെയ്യുന്നു.

Synchronous I/O

ഒരു **shared clock** ഉപയോഗിക്കുന്നു.

Asynchronous I/O

Handshake / Ready-Busy signals ഉപയോഗിക്കുന്നു.

വ്യത്യസ്ത speed ഉള്ള devices-ന് ഇത് കൂടുതൽ അനുയോജ്യമാണ്.

രണ്ട് I/O Interface Types

1. **Memory-Mapped I/O**
2. **Isolated / Port-Mapped I/O**

2.3 Types of Interrupts / Priority Interrupt



Priority Interrupt

ഒരേ സമയത്ത് രണ്ടോ അതിലധികമോ devices interrupt request നൽകിയാൽ, ഏറ്റവും ഉയർന്ന **priority** ഉള്ള **request** ആദ്യം **service** ചെയ്യുന്നു.

- Fast devices → High priority
- Slow devices → Low priority

ഉദാഹരണം:

- Disk → High priority

- Keyboard → Low priority

Priority നിശ്ചയിക്കാൻ:

1. **Polling — Software**
2. **Daisy Chain / Parallel Encoder — Hardware**

എന്നിവ ഉപയോഗിക്കാം.

Interrupt Types

1. Hardware Interrupt

External devices-ൽ നിന്ന് ഉണ്ടാകുന്ന interrupt.

Maskable Interrupt

- ആവശ്യമെങ്കിൽ delay ചെയ്യാൻ കഴിയും.

Non-Maskable Interrupt

- ഉടൻ തന്നെ handle ചെയ്യണം.

2. Software Interrupt

Program അല്ലെങ്കിൽ system-ൽ നിന്ന് ഉണ്ടാകുന്ന interrupt.

Normal

- System call / Interrupt instruction.

Exception

- Unexpected condition.
- ഉദാഹരണം: **Divide by Zero**

Interrupt Flow

Device → Priority Logic → CPU → Save PC → ISR → Return

2.4 Direct Memory Access — DMA ★★★

DMA എന്താണ്?

DMA (Direct Memory Access) ഉപയോഗിച്ച് CPU ഓരോ byte-ഉം സ്വയം transfer ചെയ്യാതെ, ഒരു block of data I/O device-നും Main Memory-ക്കും ഇടയിൽ നേരിട്ട് transfer ചെയ്യാൻ കഴിയും.

ഇവിടെ **DMA Controller (DMAC)** bus master ആയി പ്രവർത്തിക്കുന്നു.

DMA ആവശ്യമായത് എന്തുകൊണ്ട്?

Programmed I/O വളരെ slow ആണ്, പ്രത്യേകിച്ച്:

- Disk
- Network
- Multimedia

പോലുള്ള high-speed data transfer ആവശ്യമായ സാഹചര്യങ്ങളിൽ.

DMA Working

1. Device / CPU DMA request നൽകുന്നു.
2. **DMAC bus request** നൽകുന്നു.
3. CPU **bus grant** നൽകുന്നു.
4. CPU starting address, count, direction എന്നിവ set ചെയ്യുന്നു.
5. DMAC Memory ↔ Device data transfer നടത്തുന്നു.
6. Count zero ആകുമ്പോൾ bus release ചെയ്യുന്നു.
7. CPU-വിന് interrupt നൽകുന്നു.

DMA Modes

1. Burst Mode

- മുഴുവൻ block transfer പൂർത്തിയാകുന്നതുവരെ bus കൈവശം വരുന്നു.

2. Cycle Stealing

- ഒരു word transfer ചെയ്ത ശേഷം bus CPU-വിന് തിരികെ നൽകുന്നു.

3. Transparent Mode

- CPU idle ആയിരിക്കുമ്പോൾ മാത്രം bus ഉപയോഗിക്കുന്നു.

Advantages

- Bulk data transfer വേഗത്തിൽ നടക്കുന്നു.

- CPU free ആകുന്നു.
 - Throughput വർദ്ധിക്കുന്നു.
 - ഓരോ byte-ക്കും interrupt നൽകേണ്ടതില്ല.
 - ഒരു block-ന് ഒരു interrupt മതിയാകും.
-

ബ്ലോക്ക് 3: Parallel Computer Structures

3.1 Parallel vs Serial Computing ★★

Serial Computing

1. ഒരു **Processing Element (PE)** മാത്രം ഉപയോഗിക്കുന്നു.
2. Instructions ഒന്നിന് ശേഷം മറ്റൊന്ന് execute ചെയ്യുന്നു.
3. വലിയ problems-ന് slow ആണ്.
4. ഉദാഹരണം: **Uniprocessor / SISD**

Parallel Computing

1. നിരവധി Processing Elements ഉപയോഗിക്കുന്നു.
 2. ഒരേ സമയം നിരവധി operations നടത്തുന്നു.
 3. വലിയ problems-ന് കൂടുതൽ വേഗത നൽകുന്നു.
 4. ഉദാഹരണം:
 - Multiprocessor
 - Array Processor
-

3.2 Flynn's Classification ★★★

M. J. Flynn (1966) computers-നെ **Instruction Streams × Data Streams** എന്ന അടിസ്ഥാനത്തിൽ നാല് വിഭാഗങ്ങളായി classified ചെയ്തു.

1. SISD

Single Instruction, Single Data

- ഒരു instruction stream

- ഒരു data stream
- സാധാരണ uniprocessor system.

ഉദാഹരണം: Conventional PC

2. SIMD

Single Instruction, Multiple Data

ഒരു Control Unit പല Processing Elements-ലേക്കും ഒരേ instruction നൽകുന്നു.

ഉദാഹരണങ്ങൾ:

- Array Processor
- ILLIAC IV
- GPU-style SIMD

3. MISD

Multiple Instruction, Single Data

- Multiple instructions
- Single data stream
- Commercial systems-ൽ വളരെ rare ആണ്.

4. MIMD

Multiple Instruction, Multiple Data

- Multiple instructions
- Multiple data streams
- നിരവധി processors വ്യത്യസ്ത instructions execute ചെയ്യാം.

ഉദാഹരണങ്ങൾ:

- Multicore systems
- Multiprocessors
- Clusters

Conclusion

- **SISD** → Serial
- **SIMD, MIMD** → Practical parallel computing

- **MISD** → സാധാരണ ഉപയോഗത്തിൽ അപൂർവ്വം.
-

UMA vs NUMA ★★

UMA — Uniform Memory Access

1. എല്ലാ processors-നും shared memory access ചെയ്യാൻ **equal access time** ആണ്.
2. സാധാരണ **SMP systems**ൽ ഉപയോഗിക്കുന്നു.
3. Program ചെയ്യാൻ എളുപ്പമാണ്.
4. Bus/Memory contention കാരണം വലിയ scale-ലേക്ക് പോകാൻ ബുദ്ധിമുട്ടാണ്.

NUMA — Non-Uniform Memory Access

1. Local memory-യുടെ access remote memory-യേക്കാൾ വേഗത്തിലാണ്.
2. Scalable multiprocessor systems-ൽ ഉപയോഗിക്കുന്നു.
3. Programming-ൽ memory locality പരിഗണിക്കണം.
4. UMA-യേക്കാൾ കൂടുതൽ scale ചെയ്യാൻ കഴിയും.

ചുരുക്കത്തിൽ:

UMA → Simple but limited scalability

NUMA → More scalable

3.3 Pipelining ★★

Pipelining

Successive instructions-ന്റെ execution stages overlap ചെയ്ത് പ്രവർത്തിക്കുന്ന രീതിയാണ് **Pipelining**.

ഇത് ഒരു **assembly line** പോലെയാണ്.

Typical Pipeline Stages

Fetch → **Decode** → **Execute** → **Memory** → **Write-back**

Pipeline fill ആയ ശേഷം ഏകദേശം **ഒരു instruction ഒരു cycle-ലും complete** ചെയ്യാൻ കഴിയും.

Processor Cycle

tp = ഒരു stage advance ചെയ്യാൻ വേണ്ട സമയം.

ഇത് സാധാരണ:

Slowest stage + Latch time

ആണ്.

k stages, n tasks ഉള്ളപ്പോൾ:

$$\text{Time} \approx [k + (n - 1)] \times tp$$

ILP — Instruction Level Parallelism

ILP ആവശ്യമായത്:

- Performance വർദ്ധിപ്പിക്കാൻ
- Functional units കൂടുതൽ ഉപയോഗപ്പെടുത്താൻ
- Throughput വർദ്ധിപ്പിക്കാൻ
- Independent instructions schedule ചെയ്യാൻ

Clock speed മാത്രം performance നിർണ്ണയിക്കുന്നില്ല.

Pipeline Bubble

Data, Control, Structural hazard എന്നിവ കാരണം pipeline-ൽ ഉണ്ടാകുന്ന **empty/stall cycle** ആണ് Pipeline Bubble.

Types

1. Arithmetic Pipeline
2. Instruction Pipeline
3. Processor Pipeline

3.4 Performance Measures of Vector Processing ★★

Overheads

1. Setup Time

- Vector-നെ Functional Unit-ലേക്ക് route ചെയ്യാൻ വേണ്ട സമയം.

2. Flushing Time

- Pipeline-ൽ നിന്ന് ആദ്യ result പുറത്തുവരുന്നതുവരെ വേണ്ട സമയം.

Four Measures

1. Improve the Vector Instruction

- Memory accesses കുറയ്ക്കുക.
- Functional units കൂടുതൽ കാര്യക്ഷമമായി ഉപയോഗിക്കുക.

2. Integrate Scalar Instructions

- Similar scalar instructions group ചെയ്യുക.
- Pipeline വീണ്ടും വീണ്ടും reconfigure ചെയ്യേണ്ട സാഹചര്യം കുറയ്ക്കുന്നു.

3. Algorithm

- Long vectors expose ചെയ്യുന്ന algorithms തിരഞ്ഞെടുക്കുക.

4. Vectorising Compiler

- High-level code-ൽ നിന്ന് parallelism കണ്ടെത്തി ഉപയോഗിക്കുന്നു.

A → L → O → M

Supercomputers-ന്റെ performance അളക്കാൻ **FLOPS / Megaflops** ഉപയോഗിക്കുന്നു.

ശ്ലോക്ക് 4: Operating Systems

4.1 Process Management ★★

Operating System-ന്റെ പ്രധാന process-management duties:

1. Create / Delete Processes

- PCB allocate ചെയ്യുന്നു.
- Program load ചെയ്യുന്നു.
- PID assign ചെയ്യുന്നു.

2. CPU Scheduling

ഇവ ഉൾപ്പെടുന്നു:

- Long-term scheduler
- Short-term scheduler
- Medium-term scheduler
- Dispatcher

3. Suspend / Resume

Process states:

- Ready
- Waiting
- Running

ആവശ്യമെങ്കിൽ swapping നടത്താം.

4. Synchronisation

Shared data corrupt ആകാതിരിക്കാൻ synchronization നടത്തുന്നു.

5. IPC — Interprocess Communication

Processes തമ്മിൽ communication നടത്താൻ:

- Pipes
- Messages
- Shared Memory
- Sockets

എന്നിവ ഉപയോഗിക്കുന്നു.

6. Termination

Process terminate ചെയ്യുമ്പോൾ:

- CPU
- Memory
- Open files

തുടങ്ങിയ resources reclaim ചെയ്യുന്നു.

PCB — Process Control Block

PCB-യിൽ process-ന്റെ:

- State
- Program Counter
- Registers
- Scheduling information
- Memory information

എന്നിവ സൂക്ഷിക്കുന്നു.

OS = Resource Manager + Programs-നും Hardware-നും ഇടയിലെ Interface

OS-ന്റെ പ്രധാന Components

1. Process
 2. Memory
 3. File
 4. Mass Storage
 5. I/O
 6. Protection / Security
-

4.1 Types of OS / Time-Sharing ★★

1. Serial Processing

- Programmer നേരിട്ട് hardware-മായി interact ചെയ്യുന്നു.
- ഒരാൾക്ക് ശേഷം മറ്റൊരാൾ.
- Modern OS-കളിൽ സാധാരണ കാണുന്നില്ല.

2. Simple Batch

- Similar jobs operator group ചെയ്യുന്നു.
- Monitor അവ sequential ആയി execute ചെയ്യുന്നു.

3. Multiprogrammed Batch

ഒരു job I/O-ക്കായി wait ചെയ്യുമ്പോൾ CPU മറ്റൊരു job-ലേക്ക് switch ചെയ്യുന്നു.

ലക്ഷ്യം: CPU busy ആയി നിലനിർത്തുക.

4. Time-Sharing

CPU ഓരോ user-നും ചെറിയ **time slice / quantum** നൽകുന്നു.

അതിനാൽ ഓരോ user-നും CPU തന്റേതുപോലെ അനുഭവപ്പെടുന്നു.

Time-Sharing Features

- Interactive response
- നിരവധി programs memory-യിൽ
- Frequent context switching
- Fair CPU sharing

Issues

- Reliability
- Security
- Integrity
- Communication

4.1 Layered Structure of OS ★★

Operating System പല layers ആയി വിഭജിച്ചിരിക്കുന്നു.

Layer N → User Interface

Layer 0 → Hardware

ഓരോ layer-നും:

- Defined inputs
- Defined outputs
- Defined functions

ഉണ്ട്.

ഒരു layer താഴെയുള്ള layers മാത്രം ഉപയോഗിക്കുന്നു.

Advantages

- Modularity
- Debugging എളുപ്പം
- Modification എളുപ്പം
- Clear abstraction
- Lower layers hardware details hide ചെയ്യുന്നു.

Simple Structure — MS-DOS

- Module boundaries നല്ലതല്ല.
- User program-ലെ ഒരു fault പോലും system crash ചെയ്യാൻ സാധ്യതയുണ്ട്.

Microkernel — Mach

Kernel-ൽ പ്രധാനമായും:

- Address spaces
- Threads
- IPC

എന്നിവ മാത്രം നിലനിർത്തുന്നു.

മറ്റ് services user space-ൽ പ്രവർത്തിക്കുകയും messages വഴി communicate ചെയ്യുകയും ചെയ്യുന്നു.

Advantages:

- കൂടുതൽ reliable
- കൂടുതൽ secure

4.2 OS Services ★★

Operating System നൽകുന്ന പ്രധാന services:

1. **User Interface** — CLI / GUI / AUI
2. **Program Execution** — Program load ചെയ്ത് process create ചെയ്യുന്നു.
3. **Resource Allocation** — CPU, Memory, Devices allocate ചെയ്യുന്നു.
4. **I/O Operations**

5. **File-System Manipulation**

6. **Communication — IPC**

7. **Error Detection**

8. **Accounting**

9. **Protection and Security**

Process Control System Calls

fork() → Child process create ചെയ്യുന്നു.

exec() → Process image replace ചെയ്യുന്നു.

wait() → Parent process wait ചെയ്യുന്നു.

exit() → Process terminate ചെയ്യുന്നു.

4.3 Process vs Program ★★

Program

1. **Passive entity** ആണ്.
2. Disk-ൽ stored ആയ instructions-ന്റെ set ആണ്.
3. Code മാത്രം അടങ്ങിയിരിക്കുന്നു.
4. File delete ചെയ്യുന്നതുവരെ നിലനിൽക്കും.
5. ഒരു program file ആയിരിക്കും.

Process

1. **Active entity** ആണ്.
2. Execution-ൽ ഉള്ള program ആണ്.
3. Code + PC + Registers + Stack + Data + PCB എന്നിവ ഉൾപ്പെടുന്നു.
4. Create ചെയ്യപ്പെടുകയും schedule ചെയ്യപ്പെടുകയും wait ചെയ്യുകയും അവസാനം terminate ചെയ്യുകയും ചെയ്യുന്നു.
5. ഒരേ program-ൽ നിന്ന് പല processes ഉണ്ടാകാം.

Example:

Disk-ൽ ഉള്ള `Word.exe` → **Program**

Word ഇറക്കുമതി → **Process**

4.3 Process State Diagram ★★

Multiprogramming OS-ൽ ഒരു process പല states-യുടെ കടന്നുപോകുന്നു.

Five States

1. New

- Process create ചെയ്യുന്നു.
- PCB allocate ചെയ്യുന്നു.

2. Ready

- CPU കാത്തിരിക്കുന്ന process.
- Ready Queue-ൽ ഉണ്ടായിരിക്കും.

3. Running

- CPU-ൽ execute ചെയ്യുന്ന process.

4. Waiting / Blocked

- I/O അല്ലെങ്കിൽ ഒരു event കാത്തിരിക്കുന്നു.

5. Terminated

- Process execution പൂർത്തിയായി.
- Resources reclaim ചെയ്യുന്നു.

Transitions

New → Ready

- Process admitted.

Ready → Running

- Short-term scheduler / Dispatcher.

Running → Ready

- Interrupt / Time-slice end.

Running → Waiting

- I/O അല്ലെങ്കിൽ event wait.

Waiting → Ready

- I/O / event complete.

Running → Terminated

- Exit.

Context Switch

പഴയ process-ന്റെ:

- PC
- Registers
- മറ്റ് state വിവരങ്ങൾ

PCB-യിൽ save ചെയ്യുകയും പുതിയ process-ന്റെ PCB load ചെയ്യുകയും ചെയ്യുന്നു.

Schedulers

Long-term scheduler

→ ഏത് job memory-യിൽ പ്രവേശിക്കണം എന്ന് തീരുമാനിക്കുന്നു.

Short-term scheduler

→ Ready processes-ൽ നിന്ന് CPU ലഭിക്കുന്നത് ആരാണെന്ന് തീരുമാനിക്കുന്നു.

Medium-term scheduler

→ Swapping നിയന്ത്രിക്കുന്നു.

4.3 Priority vs Non-Priority / Preemptive vs Non-Preemptive ★★

Non-Preemptive / Non-Priority Scheduling

1. Running process CPU പിടിച്ചുനിൽക്കും.
2. Process finish ചെയ്യുകയോ I/O-ക്ക് block ചെയ്യുകയോ ചെയ്യുന്നതുവരെ CPU മാറ്റില്ല.
3. Examples:
 - FCFS

- SJF
4. Simple ആണ്.
 5. Context switches കുറവാണ്.
 6. **Disadvantage:** Convoy effect; important processes wait ചെയ്യേണ്ടിവരാം.

Preemptive / Priority Scheduling

1. Running process-ൽ നിന്ന് CPU forcefully എടുത്തുമാറ്റാം.
2. Higher priority process വന്നാലോ, shorter remaining time ഉണ്ടായാലോ, time quantum തീർന്നാലോ switch ചെയ്യും.
3. Examples:
 - SRTF
 - Round Robin
4. Urgent / interactive work-ന് better response.
5. **Disadvantage:** Context switches കൂടുതലാണ്.
6. Low-priority processes starvation അനുഭവിക്കാം.

Summary

Non-preemptive → CPU forcefully എടുത്തുമാറ്റില്ല.

Preemptive → CPU forcefully എടുത്തുമാറ്റാം.

Important Formulas

$$TAT = CT - AT$$

$$WT = Start - AT$$

അല്ലെങ്കിൽ:

$$WT = TAT - BT$$

$$\text{Average WT} = \text{Sum of WT} / n$$

Numerical problems-ൽ എപ്പോഴും Gantt Chart വരയ്ക്കുക.

FCFS Numerical ★★

FCFS = First Come First Serve

Arrival order അനുസരിച്ച് processes execute ചെയ്യുന്നു.

Example

P1(0,4), P2(2,5), P3(3,3), P4(4,2), P5(5,1)

Order

P1 → P2 → P3 → P4 → P5

Gantt Chart

0-4 P1 | 4-9 P2 | 9-12 P3 | 12-14 P4 | 14-15 P5

Waiting Time

- P1 = 0
- P2 = 2
- P3 = 6
- P4 = 8
- P5 = 9

Average WT = 25 / 5 = 5

SJF Numerical ★★ ★

SJF = Shortest Job First

Ready ആയിരിക്കുന്ന processes-ൽ നിന്ന് ഏറ്റവും ചെറിയ burst time ഉള്ള process ആദ്യം തിരഞ്ഞെടുക്കുന്നു.

Example:

P1(1,7), P2(2,5), P3(3,1), P4(4,2), P5(5,8)

Execution

- P1: 1-8
- P3: 8-9
- P4: 9-11
- P2: 11-16

- P5: 16–24

Waiting Time

0, 9, 5, 5, 11

Average WT = $30 / 5 = 6$

ബ്ലോക്ക് 5: Process Synchronization

5.1 Interprocess Communication — IPC ★★ ★

Processes തമ്മിൽ communication നടത്തുന്നതിനെയാണ് **IPC** എന്ന് പറയുന്നത്.

രണ്ട് IPC Models

1. Shared Memory
2. Message Passing

IPC Methods

1. Pipes

- Unidirectional.
- Related processes-ന് ഉപയോഗിക്കുന്നു.
- ഉദാഹരണം: Parent–Child.

2. Named Pipes — FIFO

- Bidirectional.
- Unrelated processes-നും ഉപയോഗിക്കാം.
- Processes pipe-ന്റെ പേര് അറിയണം.

3. Message Queues

- Messages receiver വായിക്കുന്നതുവരെ store ചെയ്യുന്നു.
- Sender-നും receiver-നും ഒരേ സമയം active ആയിരിക്കേണ്ടതില്ല.

4. Shared Memory

- ഏറ്റവും വേഗതയേറിയ IPC methods-ൽ ഒന്നാണ്.
- Synchronization-ന് Semaphore / Mutex ആവശ്യമാണ്.

5. Semaphores

- Synchronisation-നായി integer-based wait/signal operations ഉപയോഗിക്കുന്നു.
- Bulk data transfer-നല്ല.

6. Sockets

- Network-based client-server IPC.

Dining Philosophers Problem ★★

- അഞ്ച് philosophers ഉണ്ട്.
- അഞ്ച് chopsticks ഉണ്ട്.
- ഓരോ philosopher-നും ഭക്ഷണം കഴിക്കാൻ രണ്ട് chopsticks ആവശ്യമാണ്.

എല്ലാവരും ഒരേസമയം **left chopstick** എടുത്താൽ, ഓരോരുത്തരും **right chopstick** കാത്തിരിക്കും.

അങ്ങനെ എല്ലാവരും forever wait ചെയ്യുന്ന അവസ്ഥ ഉണ്ടാകും.

Four Conditions

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Solutions

1. രണ്ട് chopsticks-ഉം free ആണെങ്കിൽ മാത്രം എടുക്കുക.
2. Odd/Even philosophers വ്യത്യസ്ത order-ൽ chopsticks എടുക്കുക.
3. ഒരേസമയം maximum നാല് philosophers മാത്രം try ചെയ്യാൻ അനുവദിക്കുക.

5.4 Deadlock ★★

Deadlock എന്താണ്?

ഒരു set of processes **forever blocked** ആയിരിക്കുന്ന അവസ്ഥയാണ് Deadlock.

ഓരോ process-ും ഒരു resource കൈവശം വച്ചിരിക്കുകയും മറ്റൊരു process കൈവശം വച്ചിരിക്കുന്ന resource കാത്തിരിക്കുകയും ചെയ്യുന്നു.

Deadlock-ന്റെ നാല് Conditions

എല്ലാ നാല് conditions-ഉം ഒരേസമയം ഉണ്ടായിരിക്കണം:

1. **Mutual Exclusion**
2. **Hold and Wait**
3. **No Preemption**
4. **Circular Wait**

ഇവയിൽ ഏതെങ്കിലും ഒരു condition break ചെയ്താൽ **Deadlock** ഉണ്ടാകില്ല.

Example

- P1 → Tape T കൈവശം ഉണ്ട്; Printer Pr വേണം.
- P2 → Printer Pr കൈവശം ഉണ്ട്; Tape T വേണം.

ഇത് **Circular Wait** ഉണ്ടാക്കുന്നു.

Hold and Wait

Process ഒരു resource കൈവശം വച്ചുകൊണ്ട് മറ്റൊരു resource-നായി wait ചെയ്യുന്ന അവസ്ഥ.

Prevention

- എല്ലാ resources-ഉം ഒരുമിച്ച് request ചെയ്യുക.
- അല്ലെങ്കിൽ പുതിയ resource request ചെയ്യുന്നതിന് മുമ്പ് കൈവശമുള്ള resources release ചെയ്യുക.

5.4 Banker's Algorithm ★★

Banker's Algorithm = Deadlock Avoidance Algorithm

പ്രധാന terms:

- **Max**
- **Allocation**
- **Need**

- Available

Formula

Need = Max – Allocation

Process P_i Resource Request ചെയ്യുമ്പോൾ

1. Request $\leq$ Need?

അല്ലെങ്കിൽ → Error.

2. Request $\leq$ Available?

അല്ലെങ്കിൽ → Process wait ചെയ്യണം.

3. Tentatively Allocate

- Available കുറയ്ക്കുക.
- Allocation കൂട്ടുക.
- Need കുറയ്ക്കുക.

4. Safety Algorithm Run ചെയ്യുക

Safe sequence ലഭിച്ചാൽ:

→ Request grant ചെയ്യുക.

Safe sequence ലഭിച്ചില്ലെങ്കിൽ:

→ Allocation undo ചെയ്ത് process wait ചെയ്യുക.

Safe State

നിലവിലുള്ള available resources ഉപയോഗിച്ചും മുമ്പ് finish ചെയ്യുന്ന processes release ചെയ്യുന്ന resources ഉപയോഗിച്ചും എല്ലാ processes-നും finish ചെയ്യാൻ കഴിയുന്ന ഒരു order ഉണ്ടെങ്കിൽ അത് Safe State ആണ്.

ഇത് ഒരു banker-നെപ്പോലെയാണ്. എല്ലാ customers-ന്റെയും maximum requirement നിറവേറ്റാൻ കഴിയാത്ത വിധം പണം loan ആയി നൽകില്ല.

ബ്ലോക്ക് 6: Memory Management and File Systems

6.1 Logical vs Physical Address ★★

Logical Address

1. CPU generate ചെയ്യുന്നു.
2. **Virtual Address** എന്നും വിളിക്കുന്നു.
3. Program കാണുന്ന address ആണ്.

Physical Address

1. RAM-ലെ യഥാർത്ഥ location ആണ്.
2. **Real Address** എന്നും വിളിക്കുന്നു.
3. MMU translation വഴി ലഭിക്കുന്നു.

Formula

Physical Address = Logical Address + Base

ഇത് **Limit** ഉപയോഗിച്ച് check ചെയ്യുന്നു.

MMU

MMU = Memory Management Unit

Address binding നടക്കാവുന്ന സമയങ്ങൾ:

1. Compile time
2. Load time
3. Execution time

6.1 Contiguous Allocation / Best-Fit ★★

Contiguous Allocation

ഒരു process-ന് consecutive memory locations നൽകുന്ന രീതിയാണ് Contiguous Allocation.

Fixed അല്ലെങ്കിൽ Dynamic partitions ഉപയോഗിക്കാം.

Disadvantages

Internal Fragmentation

→ Partition-ന്റെ ഉള്ളിൽ ഉപയോഗിക്കാതെ കിടക്കുന്ന memory.

External Fragmentation

→ Memory-ൽ പല ചെറിയ scattered holes ഉണ്ടാകുന്നത്.

Placement Methods

1. First-fit
2. Next-fit
3. Best-fit
4. Worst-fit

Best-Fit

Process-ന് മതിയായ വലുപ്പമുള്ള ഏറ്റവും ചെറിയ hole തിരഞ്ഞെടുക്കുന്നു.

Advantage:

- വലിയ hole unnecessary ആയി split ചെയ്യുന്നത് ഒഴിവാക്കുന്നു.

Disadvantages:

- വളരെ ചെറിയ leftover holes ഉണ്ടാകാം.
- Search കൂടുതൽ സമയം എടുക്കാം.

Compaction

Processes-നെ memory-യിൽ ഒരമിച്ച് slide ചെയ്ത് ഒരു വലിയ free hole ഉണ്ടാക്കുന്നു.

Advantage: External fragmentation കുറയ്ക്കുന്നു.

Disadvantage: Costly ആണ്.

6.2 Paging vs Segmentation ★★★

Paging

1. Process fixed-size **pages** ആയി വിഭജിക്കുന്നു.
2. Memory same-size **frames** ആയി വിഭജിക്കുന്നു.
3. Address = **Page Number + Offset**
4. Page Table → Page-നെ Frame-ലേക്ക് map ചെയ്യുന്നു.
5. External fragmentation ഇല്ല.

6. Last page കാരണം Internal fragmentation ഉണ്ടാകാം.

7. User സാധാരണ pages നേരിട്ട് കാണുന്നില്ല.

Segmentation

1. Program variable-size logical **segments** ആയി വിഭജിക്കുന്നു.

2. ഉദാഹരണം:

- Code
- Stack
- Modules

3. Address = **Segment Number + Offset**

4. Segment Table → **Base + Limit** സൂക്ഷിക്കുന്നു.

5. Internal fragmentation ഇല്ല.

6. External fragmentation ഉണ്ടാകാം.

7. Programmer-ന്റെ logical view-നോട് കൂടുതൽ match ചെയ്യുന്നു.

8. Whole segments എളുപ്പത്തിൽ share ചെയ്യാം.

Page Table

Logical page number → Physical frame number ആയി map ചെയ്യുന്നു.

അതുകൊണ്ട് process-ന് **non-contiguous RAM**-ൽ ഇരിക്കാനാകും.

Page Fault

ഒരു process-ന് ആവശ്യമായ page address space-ൽ ഉണ്ടെങ്കിലും **RAM-ൽ ഇല്ലെങ്കിൽ Page Fault** സംഭവിക്കുന്നു.

OS ആ page RAM-ലേക്ക് load ചെയ്ത് instruction വീണ്ടും restart ചെയ്യുന്നു.

Demand Paging

Page ആവശ്യപ്പെടുമ്പോൾ മാത്രം memory-ലേക്ക് load ചെയ്യുന്നു.

Advantages:

- ഉപയോഗിക്കാത്ത pages disk-ൽ തന്നെ തുടരും.
- കൂടുതൽ multiprogramming സാധ്യമാകും.

- RAM-നേക്കാൾ വലിയ programs പ്രവർത്തിപ്പിക്കാൻ കഴിയും.

Thrashing

വളരെയധികം paging നടക്കുകയും വളരെ കുറച്ച് useful work മാത്രം നടക്കുകയും ചെയ്യുന്ന അവസ്ഥയാണ് **Thrashing**.

6.3 Page Replacement ★★★

Free frame ഇല്ലാത്തപ്പോൾ ഏത് page replace ചെയ്യണം എന്ന് തിരഞ്ഞെടുക്കണം.

1. FIFO — First In First Out

- ഏറ്റവും പഴയ page replace ചെയ്യുന്നു.
- **Belady's Anomaly** ഉണ്ടാകാം.
- കൂടുതൽ frames നൽകിയാലും ചിലപ്പോൾ കൂടുതൽ page faults ഉണ്ടാകാം.

2. Optimal

- Future-ൽ ഏറ്റവും വൈകി ഉപയോഗിക്കുന്ന page replace ചെയ്യുന്നു.
- ഏറ്റവും നല്ല fault rate നൽകുന്നു.
- Future knowledge ആവശ്യമാണ്.

3. LRU — Least Recently Used

- ഏറ്റവും കാലമായി ഉപയോഗിക്കാത്ത page replace ചെയ്യുന്നു.

Method

- ആദ്യം empty frames fill ചെയ്യുക.
- Miss വന്നാൽ algorithm rule അനുസരിച്ച് page replace ചെയ്യുക.
- **Hit = Page fault അല്ല.**

Practised Reference String

3 Frames:

9, 2, 3, 4, 1, 2, 3, 5, 1, 0, 5, 9, 9, 0, 1

Results:

- **FIFO = 11 faults**

- **Optimal = 8 faults**
- **LRU = 12 faults**

Exam-ൽ ഓരോ reference-നും ശേഷം frames-ന്റെ അവസ്ഥയും **Fault / Hit** എന്നും കാണിക്കുക.

6.4 File Allocation Methods ★★

File allocation-ന്റെ മൂന്ന് പ്രധാന methods:

1. Contiguous Allocation

File-ന്റെ blocks consecutive ആയി store ചെയ്യുന്നു.

Directory-ൽ:

- Start
- Length

എന്നിവ store ചെയ്യുന്നു.

Advantages:

- Simple
- Fast sequential access
- Fast direct access

Disadvantages:

- External fragmentation
 - File size പലപ്പോഴും creation സമയത്ത് തന്നെ declare ചെയ്യണം.
-

2. Linked Allocation

File blocks ഒരു linked list ആയി ബന്ധിപ്പിക്കുന്നു.

Directory ആദ്യ block-ലേക്ക് point ചെയ്യുന്നു.

Advantages:

- External fragmentation ഇല്ല.
- File വളർത്താൻ എളുപ്പമാണ്.

Disadvantages:

- Pointer overhead.
 - Mainly sequential access.
 - Pointer നഷ്ടപ്പെട്ടാൽ file truncate ആകാം.
-

3. Indexed Allocation

ഒരു **Index Block**-ൽ file-ന്റെ എല്ലാ block addresses-ഉം സൂക്ഷിക്കുന്നു.

Directory → Index Block-ലേക്ക് point ചെയ്യുന്നു.

Advantages:

- Direct access സാധ്യമാണ്.
- External fragmentation ഇല്ല.

Disadvantages:

- Index block-ന് അധിക storage വേണം.
 - വലിയ files-ന് multilevel indexing ആവശ്യമായി വരാം.
-

Exam-ന് ഏറ്റവും പ്രധാനപ്പെട്ട 15-mark topics

ഈ PDF അനുസരിച്ച് പ്രത്യേകമായി ശ്രദ്ധിക്കേണ്ടവ:

1. **Functional Units of Computer**
2. **Addressing Modes**
3. **DMA**
4. **Flynn's Classification**
5. **Pipelining**
6. **Process State Diagram**
7. **FCFS Numerical**
8. **SJF Numerical**
9. **IPC + Dining Philosophers**
10. **Deadlock + Banker's Algorithm**
11. **Paging vs Segmentation**

12. Page Replacement — FIFO, Optimal, LRU

13. File Allocation Methods

ഇവ **Malayalam explanation + English technical terms** ആയി പഠിച്ചാൽ exam എഴുതാൻ എളുപ്പമായിരിക്കും.